



نظري

3



13



1560 sp

كلية الهندسة المعلوماتية

السنة الثالثة

HTTP Cookies, URL
Structure, and HTTP Caching

د. غاندي هسام

Programming
Languages



RB Informatics; 29/10/2025

لغات البرمجة

ملفات تعريف الارتباط (Cookies):

Cookie: هو جزء صغير من البيانات يرسله الـ server إلى الـ browser.

يقوم المتصفح بتخزين هذه الملفات، وإنشاء ملفات جديدة، وتعديل الموجودة، وإرسالها مرة أخرى إلى نفس الخادم في الطلبات اللاحقة.

- **cookies** تمكّن تطبيقات الويب من تخزين كمية صغيرة من البيانات و الاحتفاظ بمعلومات عن حالة المستخدم (**state information**) لأن بروتوكول HTTP بطبيعته **stateless**، أي أنه لا يتذكر أي شيء بين الطلبات.
- التوضيح العلمي:
عند كل مرة يرسل فيها المتصفح طلباً جديداً إلى الخادم، لا يعرف الخادم ما إذا كان هذا الطلب من نفس المستخدم السابق ام لا، لان HTTP لا يحتفظ بأي **session state**.
- وهنا يأتي دور الـ **Cookies** فهي تربط الطلبات المختلفة بنفس المستخدم ، مما يسمح للخادم بإرسال ردود شخصية (**personalized response**) أو عامة (**generic response**) حسب الحالة.

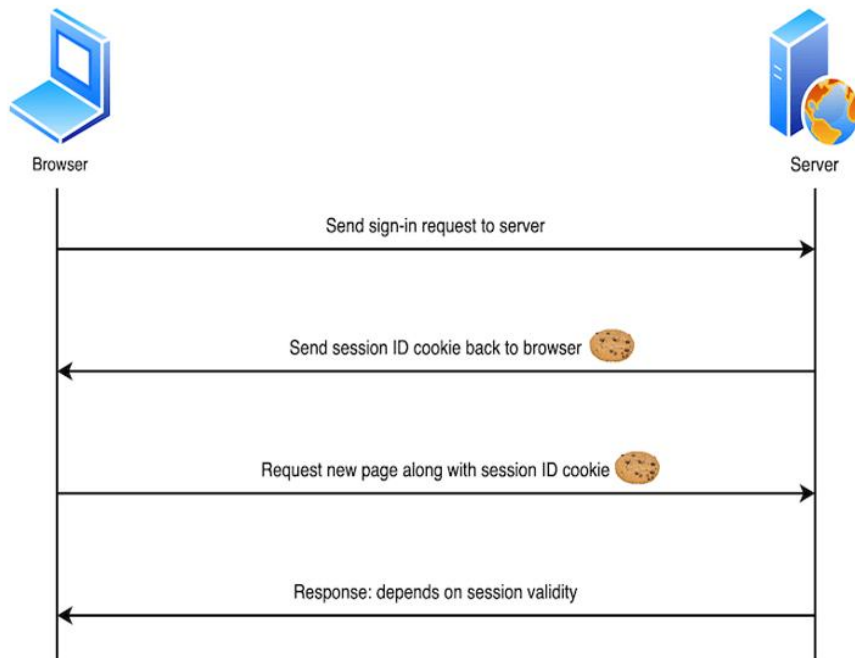
استخدامات الـ Cookies:

- **Session Management:** حفظ حالة تسجيل الدخول، سلة التسوق، نتائج اللعبة، او أي بيانات تخص جلسة المستخدم.
- **Personalization:** تخزين تفضيلات المستخدم مثل اللغة (Language preference) او النمط (theme).
- **Tracking:** تسجيل و تحليل سلوك المستخدم (User behavior) لأغراض تحليلية او إعلانية.



مثال بسيط: نظام تسجيل دخول (sign-in system) حيث:

1. المستخدم يرسل بيانات تسجيل الدخول (credentials) الى الخادم (عادةً من خلال نموذج from)
2. الخادم يتحقق من البيانات: اذا كانت صحيحة ، يرسل استجابة تحتوي على Cookie فيها session ID ، و هذا المعرف (session ID) يدل على ان المستخدم أصبح "مسجلاً الدخول".
3. بعد ذلك، عندما ينتقل المستخدم الى صفحة أخرى ضمن نفس الموقع ، المتصفح يرسل تلك ال cookie تلقائياً مع الطلب الجديد (HTTP Request)
4. الخادم يستقبل الطلب و معه session ID (ال id الموجودة ضمن ال cookie)، و يتحقق منه:
 - إذا كان ما زال صالحاً: يرسل صفحة شخصية (مثل "welcome ,Ahmad").
 - إذا لم يكن صالحاً: يحذف الصفحة و يعرض "sign in again" أو رسالة "Access Denied".



:Creating, removing, and updating cookies

بعد أن يتلقى الخادم (Server) طلباً من المتصفح (HTTP Request)، يمكنه أن يرسل في الاستجابة (HTTP Response) واحداً أو أكثر من الرؤوس (Headers) باسم Set-Cookie، وكل رأس منها يقوم بإنشاء Cookie جديدة في المتصفح.

HTTP

Set-Cookie: <cookie-name>=<cookie-value>

مثال:

عند إرسال ال server هذا السطر ضمن الاستجابة:

Set-Cookie: sessionId=abc123; Path=/; Secure

فهذا يعني أن المتصفح سيقوم بتخزين cookie بإسم sessionId وقيمة abc123.

❖ عند إرسال الطلبات لاحقاً:

في كل مرة يرسل فيها المتصفح طلباً جديداً إلى نفس النطاق (Domain) أو المسار (path) سيضيف تلقائياً ال cookie التي خزنها مسبقاً ضمن Header جديد اسمه cookie.

مثال:

Cookie: sessionId=abc123

وهكذا يعرف الخادم أن هذا الطلب يعود لنفس المستخدم الذي أرسل الطلب الأول.

HTTP

```
HTTP/2.0 200 OK
Content-Type: text/html
Set-Cookie: yummy_cookie=chocolate
Set-Cookie: tasty_cookie=strawberry
```

[page content]

المثال يوضح كيف يقوم الخادم (server) بإرسال أكثر من cookie في استجابة HTTP واحدة. عند استلام هذه الاستجابة يقوم المتصفح (Browser) بحفظ كلا ملفي ال cookie المذكورين فيها.

HTTP

```
GET /sample_page.html HTTP/2.0
Host: www.example.org
Cookie: yummy_cookie=chocolate; tasty_cookie=strawberry
```

- هنا المتصفح أرسل طلب GET للصفحة index.html/ على الموقع example.org
- مع هذا الطلب، أرسل أيضاً جميع الكوكيز التي تخص هذا النطاق ضمن Header واحد.
- و نلاحظ أن المعلومات ضمن ال cookie هي ذاتها التي كانت موجودة ضمن ال set-cookie، وبالتالي ال server يتذكر و يرسل ذات الصفحة.
- يفصل المتصفح بين كل cookie والأخرى بعلامة فاصلة منقوطة ;

ملاحظة هامة:

Set-Cookie: يُرسل من الخادم لإنشاء أو تعديل cookies في المتصفح.
Cookie: يُرسل من المتصفح إلى الخادم للإعلامه ببيانات الجلسة المخزنة.

Removal: defining the lifetime of a cookie

حذف ال Cookie نوعاً ما يوجد فيها خطورة.

عند إنشاء Cookie يمكننا تحديد متى تنتهي صلاحيتها أو متى تُحذف تلقائياً.

يتم ذلك من خلال خاصيتين (Attributes) داخل سطر ال Set-Cookie:

Expires: تحدد تاريخ انتهاء محدد (مثل : 1 Jan 2026)

Max-Age: تحدد المدة بالثواني قبل انتهاء صلاحية ال Cookie.

أنواع ال Cookie حسب العمر:

تُحذف عند انتهاء الوقت المحدد في Expires أو Max-Age.	Permanent Cookie
مؤقتة، تُحذف عند انتهاء جلسة المتصفح (إغلاقه)	Session Cookie

HTTP

Set-Cookie: id=a3fWa; Expires=Thu, 31 Oct 2021 07:28:00 GMT;

HTTP

Set-Cookie: id=a3fWa; Max-Age=2592000

ملاحظة: كل متصفح يعرف "انتهاء الجلسة" بطريقة مختلفة، بعضها يحذف ال Cookie مباشرة بعد الإغلاق، و بعضها يستعيدها عند إعادة الفتح (Session restore).

ملاحظة: اذا كان موقعك يقوم بعملية تسجيل دخول (Authentication) للمستخدمين، فمن الأفضل أن يعيد إنشاء (regenerate) و يرسل Session Cookies جديدة كل مرة يقوم فيها المستخدم بتسجيل الدخول بنجاح، حتى لو كان لديه Cookie قديمة بنفس الاسم. (و ذلك في الويب القديم)

الهدف من ذلك: منع نوع من الهجمات يُسمّى Session Fixation Attack وهو هجوم يستغل فيه طرف ثالث (مخترق) نفس رقم الجلسة (Session ID) لمستخدم آخر.

❖ Immediate Removal (حذف ال Cookie مباشرة):

أحياناً نريد حذف Cookie فوراً، بدون انتظار انتهاء الوقت المحدد لها.

في هذه الحالة ، الخادم (server) يرسل نفس اسم ال Cookie، لكن مع عمر يساوي صفر (Max-Age=0)

HTTP

`Set-Cookie: id=a3fWa; Max-Age=0`

❖ حذف جميع الـ Cookies لموقع واحد:

أحيانا نريد حذف كل الـ Cookies التي تخص موقعا معينا (وليس واحدة فقط)



HTTP

`Clear-Site-Data: "cookies"`

عندما يرسل الموقع هذا السطر في الاستجابة:

- يتم مسح جميع الكوكيز الخاصة بالموقع نفسه.
- و أيضا الخاصة بجميع النطاقات الفرعية (subdomains) التابعة له.

❖ تحديث الـ Cookie (Updating Cookies):

لتحديث cookie موجودة، يكفي أن يرسل الخادم من جديد نفس الاسم لكن بقيمة مختلفة.

HTTP

`Set-Cookie: id=new-value`

أيضا من جهة المتصفح، يمكن تحديث او إنشاء Cookie باستخدام JavaScript من خلال الخاصية `Document.cookie`، مثال:

JS

```
document.cookie = "yummy_cookie=chocolate";
document.cookie = "tasty_cookie=strawberry";
```

هذه الطريقة تستخدم من داخل صفحات الويب مباشرة ، لكن يجب الحذر لأنها أقل أماناً، خصوصا إذا لم يتم استخدام الخصائص `HttpOnly` أو `Secure` في الـ Cookie الأصلية.

:Cookie Security

الـ Cookies بطبيعتها مرئية للمستخدم و يمكن تعديلها بسهولة، أي شخص يستطيع الاطلاع على قيمها من داخل المتصفح، بل و حتى تغييرها يدوياً، لذلك يجب تأمينها.

“All cookie values are visible to, and can be changed by, the end user.”

لو لم تحمي الـ Cookies، يمكن لمهاجم (Attacker) أن يسرق معلومات حساسة مثل Session ID أو Token لتسجيل الدخول.

ثم يستخدمها لتسجيل الدخول كأنه المستخدم الحقيقي (وهذا يُسمى Session Hijacking اختطاف الجلسة)

:Cookie Security Attributes

أنت كمطور backend يمكنك أن ترسل الـ Cookies بطريقة آمنة و لا يمكن الوصول إليها من أطراف غير مصرح بها، وذلك باستخدام:

1. Secure Attribute

الخاصية Secure تضاف أثناء إنشاء الـ Cookie لتحديد أنها يجب أن ترسل فقط عبر اتصال آمن HTTPS (أي مشفر)

“A cookie with the Secure attribute is only sent over HTTPS.”

أي إذا حاول المتصفح إرسالها عبر HTTP العادي (غير المشفر) فلن يرسلها.

مثال: Set-Cookie: userToken=12345; Secure

الفائدة: تمنع المهاجمين من التنصت (Sniffing) على البيانات أو سرقة الـ Cookie في حال كان الاتصال غير مشفر.

2. HttpOnly Attribute

الخاصية HttpOnly تمنع الوصول إلى الـ Cookie من خلال JavaScript (أي لا يمكن قراءتها باستخدام document.cookie)

“A cookie with the HttpOnly attribute can't be accessed by JavaScript.”

مثال:

```
Set-Cookie: id=a3fWa; Expires=Thu, 21 Oct 2021 07:28:00 GMT; Secure; HttpOnly
```

الفائدة: تحمي من هجمات XSS (Cross-Site Scripting) التي تعتمد على إدخال سكريبت خبيث في الموقع لسرقة الكوكيز.

➤ لزيادة الحماية:

الـ Cookies ما ترسل لكل الصفحات في الانترنت ، بل يتم تحديد أين و متى ترسل عن طريق الخصائص:

• Domain Attribute

• Path Attribute

هذه الخصائص تحدد "نطاق الـ Cookie" (Scope of the cookie)

أي: إلى أي عناوين (URLs) يسمح بإرسالها.

❖ خاصية Domain:

تحدد اسم الخادم (Server) أو الموقع الذي يُسمح له باستقبال الـ Cookie.

“The Domain attribute specifies which server can receive a cookie.”

مثال:

```
Set-Cookie: id=a3fWa; Expires=Thu, 21 Oct 2021 07:28:00 GMT; Secure;
HttpOnly; Domain=mozilla.org
```

ملاحظة: إذا لم يتم تحديد خاصية Domain فان الـ Cookie تُرسل فقط للخادم الذي انشأها وليس إلى النطاقات الفرعية (not on its subdomains)

❖ خاصية Path:

تحدد المسار داخل عنوان الموقع (URL path) الذي يجب ان يكون موجودا بالطلب حتى تُرسل الـ Cookie معه.

“The Path attribute indicates a URL path that must exist in the requested URL in order to send the Cookie header.”

```
Set-Cookie: id=a3fWa; Expires=Thu, 21 Oct 2021 07:28:00 GMT; Secure;
HttpOnly; Path=/docs
```

ملاحظة: الرمز (%x2F) او ("/") يُعتبر فاصل المجلدات (Directory Separator) يعني أن المجلدات الفرعية (Subdirectories) تحت /docs تتطابق أيضا و تستقبل نفس الـ Cookie.

➤ تبسيط الفكرة:

تخيل أن الـ Domain مثل عنوان المبنى على الانترنت، عند تحديد Domain في الـ Cookie، كأن تقول: هذه الـ Cookie ترسل فقط لهذا المبنى وكل الفروع التابعة له.

مثال: Domain=example.com، فهي تُرسل إلى example.com وكل فروعه مثل mail.example.com و shop.example.com.

لكن إذا لم يتم تحديد الـ Domain نهائياً، فهي تُرسل فقط للموقع الأصلي الذي انشأه (بدون الفروع).
الـ Path مثل الطابق أو القسم داخل المبنى، الـ Cookie لا تذهب لجميع الغرف، بل فقط للطابق المحدد.
مثال: Path=/docs، يعني الـ Cookie تُرسل فقط للصفحات التي عناوينها تبدأ بـ /docs.



ملاحظة: كل ما كانت خصائص Domain و Path محددة أكثر ، كل ما كانت الكوكي أكثر امانا و اقل احتمال انها تُرسل لمكان غلط.

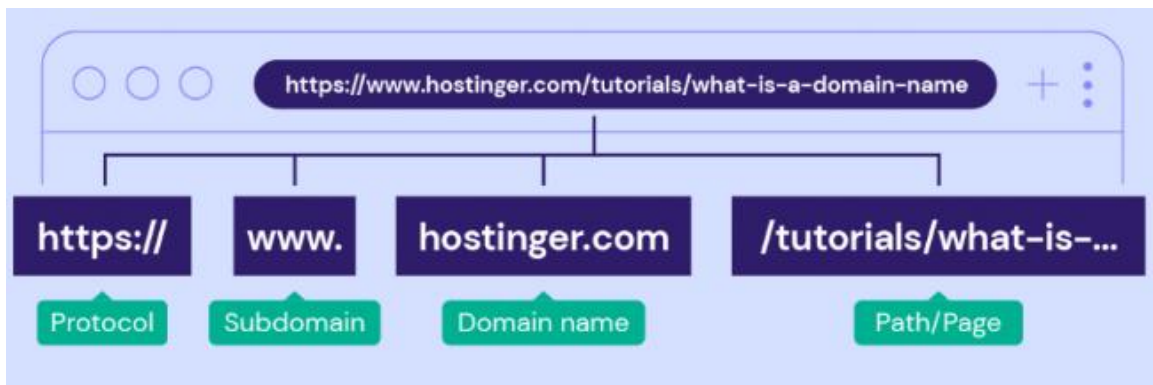
ملاحظة: من الأفضل لك كمطور backend حماية الكوكي بكافة الطرق... أي وضع كل شيء يتعلق بالحماية لتفادي الهجمات.

What is a URL:

URL (Uniform Resource Locator) هو نوع من URI (Uniform Resource Identifier) يُستخدم للوصول إلى معلومات أو ملفات موجودة على خوادم بعيدة (Remote Servers) الـ URI هو العنوان الكامل لأي مورد على الإنترنت (صفحة، صورة، فيديو، ملف...)

🔗 يتكون من عدة عناصر منها:

- Network communication protocol: البروتوكول المستخدم مثل HTTP أو HTTPS
- Subdomain: النطاق الفرعي
- Domain name: اسم الموقع
- Extension (TLD): الامتداد بعد اسم النطاق (مثل .com أو .org ...)



🔗 أهمية ال URL:

- URL منسق بشكل جيد ، يساعد في تحسين تجربة المستخدم (User Experience)
 - كما انه عامل مهم في تحسين نتائج البحث (SEO) لأن محركات البحث تفضل الروابط المنظمة و المفهومة.
- [لذلك تنظيم الروابط \(Optimizing URLs\) يساعد في رفع ترتيب الموقع في نتائج محركات البحث \(SERPs\).](#)

🔗 HTTP و HTTPS:

- HTTP: بروتوكول لنقل البيانات بين الخادم و المتصفح بدون تشفير (غير امن)
- HTTPS: بروتوكول لنقل البيانات بين الخادم و المتصفح بشكل مشفر (امن و افضل ل SEO) باستخدام TLS و يستخدم المنفذ رقم 443.

:Domain Name ↩

هو ما يكتبه المستخدم في شريط العنوان للوصول إلى الموقع، يتكون من Site name و Extension (.com, .org, .net, .sy)

<https://www.hostinger.com>

:Domain Extension = Top Level Domain (TLD) ↩

<https://www.hostinger.com>

أنواعه:

- gTLD (Generic): مثل .com - .org - .net
- ccTLD (Country Code): امتداد يدل على دولة مثل .sy (سوريا) - .fr (فرنسا) - .jp (اليابان)
- sTLD (Sponsored): امتداد خاص بمنظمة أو جهة معينة مثل .travel - .asia
- nTLD (new gTLD): امتدادات حديثة أطلقت بعد عام 2012 ، مثل : .tech - .online - .store

❖ Path (المسار):

هو الجزء من الـ URL الذي يأتي بعد اسم النطاق (Domain) و يشير إلى موقع المورد (الملف أو الصفحة) داخل الموقع.
 “A path to the resource is the part to the right of the TLD. It’s often referred to as the folder structure of the website.”

<https://www.hostinger.com/tutorials/how-to-make-a-website>

❖ Parameters (المعاملات أو القيم):

هي القيم المتغيرة التي تضاف بعد علامة الاستفهام ؟ وتستخدم لإرسال معلومات إضافية إلى الخادم، تكتب على شكل مفاتيح وقيم (key=value).

إذا كان هناك أكثر من متغير، تفصل بينهما علامة & .

بعض الرموز: %3A << : و %2F << /

لأن ‘ / ’ و ‘ : ’ و رموز أخرى...تستخدم لأغراض أخرى.. لذلك نستخدم رموز مكافئة لها.

:Parameters Use Cases ↩

- أهم استخدامات الـ parameters:

■ Translating: تستخدم لتحديد لغة الصفحة حسب الدولة، مثل: example.com?lang=ar

■ Searching: تستخدم في مربعات البحث داخل الموقع مثل: example.com/search?query=weather

■ Filtering: لتصفية المحتوى حسب خصائص معينة (اللون , السعر , الفئة ...)

مثل: `example.com/products?color=red&price=low`

■ Paginating: لتقسيم الصفحات مثل صفحات نتائج المتجر أو المقالات

مثل: `example.com/articles?page=3`

“These parameters help organize, filter, and deliver content efficiently.”

❖ Anchor Links (الربط الداخلي):

الـ Anchor link هو الجزء الذي يلي الرمز # في نهاية الـ URL

- يُستخدم للانتقال إلى جزء معين داخل نفس الصفحة بدل تحميل صفحة جديدة.

`.hostinger.com/tutorials/how-to-make-a-website#6_Optimize_Your_Website_for_SEO_and_Conversions`

هذا الرابط ينقلك مباشرة إلى القسم ما بعد # داخل صفحة tutorial.

- في صفحات الـ HTML ، يحدّد مكان داخل الصفحة لتسهيل التصفح و خاصة في الصفحات الطويلة.
- في ملفات الفيديو أو الصوت، يمكن أن يشير إلى وقت معين (timestamp) مثلا يبدأ تشغيل الفيديو في الدقيقة 2:00

“For video or audio, the browser will find the timestamp represented by the anchor and play from there.”

⬅ HTTP Cache

هو آلية يقوم فيها المتصفح أو خادم وسيط (proxy server) بتخزين نسخة من الاستجابة (Response) السابقة لإعادة استخدامها لاحقا بدل طلبها من الخادم كل مرة، يمكنه استخدام نسخة محفوظة (cache) إذا كانت مازالت صالحة (fresh).

“The HTTP cache stores a response associated with a request and reuses it for subsequent requests.”

مثال بسيط:

لو دخل المستخدم إلى صفحة موقع لأول مرة، يحمّل المتصفح الصور و الملفات من الخادم و يحفظها مؤقتا. في المرة الثانية التي يزور فيها نفس الصفحة، يُحمّلها من الكاش بدل إعادة تنزيلها من السيرفر، و هذا يجعلها تفتح بسرعة كبيرة.

الفائدة:

- تقليل عدد الطلبات المرسلة للخادم (Reduce Server load)
- تسريع تحميل الصفحات (improve performance)
- تقليل استهلاك الانترنت (save)



Types of Cache ↩

Private Cache (التخزين المؤقت الخاص): ❖

- يرتبط بمستخدم واحد فقط، عادةً في المتصفح نفسه.
- يُستخدم لتخزين محتوى مخصص لهذا المستخدم مثل: حالة تسجيل الدخول أو إعدادات العرض أو صفحات شخصية.

HTTP

Cache-Control: private

ملاحظة مهمة: حتى لو كانت الاستجابة تحتوي على Cookie هذا لا يعني بالضرورة أنها خاصة، لأن بعض ال cookies عامة الاستخدام، لذلك يجب على الخادم تحديد ذلك صراحة باستخدام التوجيه private.

Shared Cache (التخزين المؤقت المشترك): ❖

- يوجد بين العميل والخادم، ويمكن مشاركته بين عدة مستخدمين.
- الهدف منه تخفيف الضغط على الخادم بتخزين الملفات الشائعة.
- يُستخدم غالباً في: proxy servers و CDNs(Content Delivery Networks)

أنواعه:

- Proxy caches: كاشات وسيطة خارج تحكّم المطور.
- Managed Caches: كاشات يديرها مطور الخدمة نفسه (مثل شبكات CDN)

Managed Cache: مثال على

شبكات توزيع المحتوى (CDNs) مثل: Akamai , Cloudflare تخزن الصور و الملفات الثابتة و تقدّمها من اقرب خادم للمستخدم.

Cache – Control Header ↩

Cache-control header يُستخدم للتحكم في كيفية تخزين الاستجابات و في المدة الزمنية التي تُعتبر فيها صالحة (fresh).

“The HTTP Cache-Control header holds directives that control caching in browsers and shared caches.”

خصائصه:

- يُكتب داخل كل من الطلب (request) او الاستجابة (response).
- يمكن أن يحتوي على أكثر من توجيه (directive) مفصول بفاصلة.

Cache-Control: max-age=180, public

- أسماء التوجيهات غير حساسة لحالة الأحرف (case-insensitive) ، لكن يُفضل كتابتها بحروف صغيرة.

Cache-Control Directives

توجيهات cache-control هي كلمات مفتاحية تُستخدم لتحديد مدة الصلاحية (freshness) و كيفية التعامل مع النسخ المخزنة مؤقتاً في الكاش.

“Caching directives are case-insensitive and can be combined, separated by commas.”

مثال: Cache-Control: max-age=180, public

هذا يعني أنه يمكن تخزين الاستجابة لمدة 180 ثانية و هي متاحة للجميع (ليست خاصة بمستخدم محدد)

التركيب العام للتوجيه: directive-name=argument

أهم الـ directive في الـ Responses

max-age=N

- تحدد عدد الثواني التي تبقى فيها الاستجابة صالحة (fresh)
- بعد انقضاء هذه المدة ، تعتبر قديمة (stale)
- مثال: Cache-Control: max-age=600

الاستجابة تُستخدم من الكاش لمدة 10 دقائق قبل التحقق من الخادم من جديد.

s-maxage=N

- مشابهة لـ max-age ، لكنها خاصة بالكاشات المشتركة فقط (مثل CDN أو Proxy)
- تُستخدم عندما تريد تحديد صلاحية مختلفة للكاش المشترك عن الكاش الخاص .
- مثال : Cache-Control: s-maxage=1200

تبقى صالحة في الكاش المشترك لمدة 20 دقيقة.

no-cache

- تعني انه يمكن تخزين الاستجابة في الكاش، لكن يجب التحقق من صحتها مع الخادم في كل مرة قبل استخدامها.
- **ملاحظة مهمة:** no-cache لا تعني "لا تخزن" بل تعني "خزن لكن تأكد من صحتها قبل الاستخدام"
- مثال: Cache-Control: no-cache

must-revalidate

- تعني انه بعد انتهاء مدة الصلاحية (max-age) ، لا يمكن استخدام النسخة القديمة الا بعد التحقق من الخادم
- غالباً تُستخدم مع max-age.
- مثال: Cache-Control: max-age=300, must-revalidate

يمكن استخدام النسخة المخزنة لمدة 5 دقائق فقط ، وبعدها يجب التحقق مجدداً من الخادم.

مثال: في صفحة الفيس قد يظهر منشورات

قديمة، قبل 6 أيام مثلاً، هذا يعني انها

stale

ومطور الباك لم يضع max- و no-cache

age أو أخرى، التي تمنع ظهور المنشورات

proxy-revalidate

مشابهة لـ must-revalidate لكنها خاصة فقط بالكاشات المشتركة (shared caches)

no-store

تمنع أي نوع من الكاش (سواء خاص او مشترك) من تخزين الاستجابة إطلاقاً.
تستخدم عادة مع البيانات الحساسة (مثل كلمات المرور او تفاصيل الحسابات البنكية)

مثال: Cache-Control: no-store

يعني لا يُسمح بتخزين هذه البيانات نهائياً في أي كاش.

جميع ما سبق في الـ **Response**.

أيضاً no-cache يمكن استخدامه أيضاً من جهة العميل client وليس فقط في استجابات الخادم، يعني في الـ Request

“The no-cache request directive allows clients to request the most up-to-date response even if the cache has a fresh response.”

عندما يرسل الـ browser طلباً يحتوي على: **Cache-Control: no-cache**

فهو يجبر السيرفر أن يتحقق دائماً من وجود نسخة جديدة حتى لو كانت النسخة القديمة المخزنة ما زالت fresh في الكاش.

يستخدم عند تحديث الصفحات يدوياً (reload/refresh)، أو عند التعامل مع محتوى يتغير بسرعة مثل الأخبار أو البيانات المباشرة.

فائدة:

- F5: يعمل تحديث عادي، يستخدم الملفات المخزنة مؤقتاً اذا كانت ما زالت صالحة في الكاش.
- Ctrl+F5: يعمل تحديث قوي، يتجاهل الكاش كلياً و يجبر المتصفح على جلب احدث نسخة للصفحة من السيرفر.



Fake it until you make it...

انتهت المحاضرة

<The end>